```
// Code your testbench here
// or browse Examples
module ALU (
    input  wire        clk,
    input  wire        reset,
    input  wire        en_fir,

    input  wire [15:0] data_memory_in_L,
    input  wire [15:0] data_memory_in_R,
    input  wire [7:0]  rj_L_data,
    input  wire [7:0]  rj_R_data,
    input  wire [15:0] coeff_L_data,
    input  wire [15:0] coeff_R_data,

    output wire [39:0] shift_outL,
    output wire [39:0] shift_outR,
    output wire        done_shifting
);

    wire [23:0] sign_ext_L, sign_ext_R;
    wire [23:0] adder_out_L, adder_out_R;

    wire clear, clear_reg, load_adder, load_shift, shift;
    wire [3:0] read_addr_rj_L, read_addr_rj_R;
    wire [4:0] read_addr_coeff_L, read_addr_coeff_R;
    wire [7:0] read_addr_data_L, read_addr_data_R;

    // Sign Extension
    SignExtender sign_ext (
        .datain({data_memory_in_L, data_memory_in_R}),
        .dataout_L(sign_ext_L),
        .dataout_R(sign_ext_R)
    );

    // Adder
    Adder adder (
        .dataout_L(sign_ext_L),
        .shift_outL(shift_outL[39:16]),
        .add_sub_L(1'b0),
        .adder_outL(adder_out_L),
        .dataout_R(sign_ext_R),
        .shift_outR(shift_outR[39:16]),
        .add_sub_R(1'b0),
        .adder_outR(adder_out_R)
    );

    // Shift Register
    ShiftRightRegister shift_reg (
        .clk(clk),
        .reset(reset),
        .load(load_shift),
        .adder_outL(adder_out_L),
        .adder_outR(adder_out_R),
        .shift_outL(shift_outL),
        .shift_outR(shift_outR),
        .done_shifting(done_shifting)
    );

    // Controller
    ALU_Controller controller (
        .clk(clk),
        .reset(reset),
        .en_fir(en_fir),
        .rj_L_data(rj_L_data),
        .rj_R_data(rj_R_data),
        .coeff_L_data(coeff_L_data),
        .coeff_R_data(coeff_R_data),
        .clear(clear),
        .clear_reg(clear_reg),
```

```
70          .load_adder(load_adder),
71          .load_shift(load_shift),
72          .shift(shift),
73          .read_addr_data_L(read_addr_data_L),
74          .read_addr_data_R(read_addr_data_R),
75          .read_addr_rj_L(read_addr_rj_L),
76          .read_addr_rj_R(read_addr_rj_R),
77          .read_addr_coeff_L(read_addr_coeff_L),
78          .read_addr_coeff_R(read_addr_coeff_R),
79          .done_shifting(done_shifting)
80      );
81
82  endmodule
83
84  module ALU_Controller (
85      input  wire        clk,
86      input  wire        reset,
87      input  wire        en_fir,
88      input  wire [7:0]  rj_L_data,
89      input  wire [7:0]  rj_R_data,
90      input  wire [15:0] coeff_L_data,
91      input  wire [15:0] coeff_R_data,
92      input  wire        done_shifting,
93
94      output reg         clear,
95      output reg         clear_reg,
96      output reg         load_adder,
97      output reg         load_shift,
98      output reg         shift,
99
100     output reg  [7:0]  read_addr_data_L,
101     output reg  [7:0]  read_addr_data_R,
102     output reg  [3:0]  read_addr_rj_L,
103     output reg  [3:0]  read_addr_rj_R,
104     output reg  [4:0]  read_addr_coeff_L,
105     output reg  [4:0]  read_addr_coeff_R
106 );
107
108     // FSM state definitions
109     reg [2:0] state, next_state;
110     localparam IDLE            = 3'd0,
111                LOAD_RJ         = 3'd1,
112                LOAD_COEFF      = 3'd2,
113                CALC_INDEX      = 3'd3,
114                SHIFT_DATA      = 3'd4,
115                WAIT_SHIFT_DONE = 3'd5,
116                INCREMENT       = 3'd6,
117                DONE            = 3'd7;
118
119     // Internal registers
120     reg [9:0] n_L, n_R;
121     reg [3:0] tap_L, tap_R;
122     reg [4:0] coeff_L_count, coeff_R_count;
123     reg [7:0] rj_L_value, rj_R_value;
124     reg [7:0] coeff_delay_L, coeff_delay_R;
125     reg signed [8:0] index_L, index_R;
126
127     // Sequential logic: FSM state and counters
128     always @(posedge clk or posedge reset) begin
129         if (reset) begin
130             state <= IDLE;
131             n_L <= 0; n_R <= 0;
132             tap_L <= 0; tap_R <= 0;
133             coeff_L_count <= 0; coeff_R_count <= 0;
134             rj_L_value <= 0; rj_R_value <= 0;
135             coeff_delay_L <= 0; coeff_delay_R <= 0;
136             index_L <= 0; index_R <= 0;
137         end else begin
138             state <= next_state;
```

```
139
140             case (state)
141                 LOAD_RJ: begin
142                     rj_L_value <= rj_L_data;
143                     rj_R_value <= rj_R_data;
144                 end
145
146                 LOAD_COEFF: begin
147                     coeff_delay_L <= coeff_L_data[7:0];
148                     coeff_delay_R <= coeff_R_data[7:0];
149                 end
150
151                 CALC_INDEX: begin
152                     index_L <= n_L - coeff_delay_L;
153                     index_R <= n_R - coeff_delay_R;
154                 end
155
156                 INCREMENT: begin
157                     // LEFT loop logic
158                     if (coeff_L_count < rj_L_value - 1)
159                         coeff_L_count <= coeff_L_count + 1;
160                     else begin
161                         coeff_L_count <= 0;
162                         if (tap_L < 15)
163                             tap_L <= tap_L + 1;
164                         else begin
165                             tap_L <= 0;
166                             if (n_L < 999)
167                                 n_L <= n_L + 1;
168                         end
169                     end
170
171                     // RIGHT loop logic
172                     if (coeff_R_count < rj_R_value - 1)
173                         coeff_R_count <= coeff_R_count + 1;
174                     else begin
175                         coeff_R_count <= 0;
176                         if (tap_R < 15)
177                             tap_R <= tap_R + 1;
178                         else begin
179                             tap_R <= 0;
180                             if (n_R < 999)
181                                 n_R <= n_R + 1;
182                         end
183                     end
184                 end
185             endcase
186         end
187     end
188
189     // Combinational FSM logic
190     always @(*) begin
191         // Default outputs
192         clear = 0;
193         clear_reg = 0;
194         load_adder = 0;
195         load_shift = 0;
196         shift = 0;
197
198         read_addr_rj_L = tap_L;
199         read_addr_rj_R = tap_R;
200         read_addr_coeff_L = coeff_L_count;
201         read_addr_coeff_R = coeff_R_count;
202         read_addr_data_L = 8'd0;
203         read_addr_data_R = 8'd0;
204
205         next_state = state;
206
207         case (state)
```

```
            IDLE: begin
                if (en_fir) begin
                    clear = 1;
                    clear_reg = 1;
                    next_state = LOAD_RJ;
                end
            end

            LOAD_RJ: begin
                next_state = LOAD_COEFF;
            end

            LOAD_COEFF: begin
                next_state = CALC_INDEX;
            end

            CALC_INDEX: begin
                load_adder = 1;
                read_addr_data_L = index_L[7:0];
                read_addr_data_R = index_R[7:0];
                next_state = SHIFT_DATA;
            end

            SHIFT_DATA: begin
                load_shift = 1;
                next_state = WAIT_SHIFT_DONE;
            end

            WAIT_SHIFT_DONE: begin
                if (done_shifting)
                    next_state = INCREMENT;
            end

            INCREMENT: begin
                if (n_L >= 999 && n_R >= 999)
                    next_state = DONE;
                else
                    next_state = LOAD_RJ;
            end

            DONE: begin
                next_state = IDLE;
            end
        endcase
    end

endmodule



module SignExtender (
  input  wire [31:0] datain,
  output wire [23:0] dataout_L,
  output wire [23:0] dataout_R
);
  // Sign-extend left half (bits 31:16) using bit 31
  assign dataout_L = {{8{datain[31]}}, datain[31:16]};

  // Sign-extend right half (bits 15:0) using bit 15
  assign dataout_R = {{8{datain[15]}}, datain[15:0]};
endmodule


//adder is been updated to do add on both Left and Right
module Adder (
  //Left side of the adder
  input  wire [23:0] dataout_L,
  input  wire [23:0] shift_outL,
  input  wire        add_sub_L, // 0: add, 1: subtract
```

```
    output wire [23:0] adder_outL,

    //Right side of the adder
    input  wire [23:0] dataout_R,
    input  wire [23:0] shift_outR,
    input  wire        add_sub_R, // 0: add, 1: subtract
    output wire [23:0] adder_outR
);
    assign adder_outL = add_sub_L ? (shift_outL - dataout_L) : (shift_outL + dataout_L);
    assign adder_outR = add_sub_R ? (shift_outR - dataout_R) : (shift_outR + dataout_R);
endmodule



module ShiftRightRegister (
    input  wire        clk,
    input  wire        reset,
    input  wire        load,
    input  wire [23:0] adder_outL,
    input  wire [23:0] adder_outR,
    output reg  [39:0] shift_outL,
    output reg  [39:0] shift_outR,
    output reg         done_shifting
);

    reg [5:0] shift_count;
    reg       shifting;

    always @(posedge clk or posedge reset) begin
        if (reset) begin
            shift_outL     <= 40'd0;
            shift_outR     <= 40'd0;
            shift_count    <= 6'd0;
            shifting       <= 1'b0;
            done_shifting  <= 1'b0;
        end else begin
            done_shifting <= 1'b0;

            if (load) begin
                shift_outL[39:16] <= adder_outL;
                shift_outL[15:0]  <= 16'd0;
                shift_outR[39:16] <= adder_outR;
                shift_outR[15:0]  <= 16'd0;
                shift_count       <= 6'd0;
                shifting          <= 1'b1;
            end else if (shifting) begin
                shift_outL <= {1'b0, shift_outL[39:1]};
                shift_outR <= {1'b0, shift_outR[39:1]};
                shift_count <= shift_count + 1;

                if (shift_count == 6'd31) begin
                    shifting      <= 1'b0;
                    done_shifting <= 1'b1;
                end
            end
        end
    end

endmodule


//Testbench for ALU_Top Module
// Code your testbench here
// or browse Examples
module ALU (
    input  wire        clk,
    input  wire        reset,
    input  wire        en_fir,

```

```
346         input  wire [15:0] data_memory_in_L,
347         input  wire [15:0] data_memory_in_R,
348         input  wire [7:0]  rj_L_data,
349         input  wire [7:0]  rj_R_data,
350         input  wire [15:0] coeff_L_data,
351         input  wire [15:0] coeff_R_data,
352
353         output wire [39:0] shift_outL,
354         output wire [39:0] shift_outR,
355         output wire        done_shifting
356     );
357
358         wire [23:0] sign_ext_L, sign_ext_R;
359         wire [23:0] adder_out_L, adder_out_R;
360
361         wire clear, clear_reg, load_adder, load_shift, shift;
362         wire [3:0] read_addr_rj_L, read_addr_rj_R;
363         wire [4:0] read_addr_coeff_L, read_addr_coeff_R;
364         wire [7:0] read_addr_data_L, read_addr_data_R;
365
366         // Sign Extension
367         SignExtender sign_ext (
368             .datain({data_memory_in_L, data_memory_in_R}),
369             .dataout_L(sign_ext_L),
370             .dataout_R(sign_ext_R)
371         );
372
373         // Adder
374         Adder adder (
375             .dataout_L(sign_ext_L),
376             .shift_outL(shift_outL[39:16]),
377             .add_sub_L(1'b0),
378             .adder_outL(adder_out_L),
379             .dataout_R(sign_ext_R),
380             .shift_outR(shift_outR[39:16]),
381             .add_sub_R(1'b0),
382             .adder_outR(adder_out_R)
383         );
384
385         // Shift Register
386         ShiftRightRegister shift_reg (
387             .clk(clk),
388             .reset(reset),
389             .load(load_shift),
390             .adder_outL(adder_out_L),
391             .adder_outR(adder_out_R),
392             .shift_outL(shift_outL),
393             .shift_outR(shift_outR),
394             .done_shifting(done_shifting)
395         );
396
397         // Controller
398         ALU_Controller controller (
399             .clk(clk),
400             .reset(reset),
401             .en_fir(en_fir),
402             .rj_L_data(rj_L_data),
403             .rj_R_data(rj_R_data),
404             .coeff_L_data(coeff_L_data),
405             .coeff_R_data(coeff_R_data),
406             .clear(clear),
407             .clear_reg(clear_reg),
408             .load_adder(load_adder),
409             .load_shift(load_shift),
410             .shift(shift),
411             .read_addr_data_L(read_addr_data_L),
412             .read_addr_data_R(read_addr_data_R),
413             .read_addr_rj_L(read_addr_rj_L),
414             .read_addr_rj_R(read_addr_rj_R),
```

```
        .read_addr_coeff_L(read_addr_coeff_L),
        .read_addr_coeff_R(read_addr_coeff_R),
        .done_shifting(done_shifting)
    );

endmodule

module ALU_Controller (
    input  wire        clk,
    input  wire        reset,
    input  wire        en_fir,
    input  wire [7:0]  rj_L_data,
    input  wire [7:0]  rj_R_data,
    input  wire [15:0] coeff_L_data,
    input  wire [15:0] coeff_R_data,
    input  wire        done_shifting,

    output reg         clear,
    output reg         clear_reg,
    output reg         load_adder,
    output reg         load_shift,
    output reg         shift,

    output reg  [7:0]  read_addr_data_L,
    output reg  [7:0]  read_addr_data_R,
    output reg  [3:0]  read_addr_rj_L,
    output reg  [3:0]  read_addr_rj_R,
    output reg  [4:0]  read_addr_coeff_L,
    output reg  [4:0]  read_addr_coeff_R
);

    // FSM state definitions
    reg [2:0] state, next_state;
    localparam IDLE            = 3'd0,
               LOAD_RJ         = 3'd1,
               LOAD_COEFF      = 3'd2,
               CALC_INDEX      = 3'd3,
               SHIFT_DATA      = 3'd4,
               WAIT_SHIFT_DONE = 3'd5,
               INCREMENT       = 3'd6,
               DONE            = 3'd7;

    // Internal registers
    reg [9:0] n_L, n_R;
    reg [3:0] tap_L, tap_R;
    reg [4:0] coeff_L_count, coeff_R_count;
    reg [7:0] rj_L_value, rj_R_value;
    reg [7:0] coeff_delay_L, coeff_delay_R;
    reg signed [8:0] index_L, index_R;

    // Sequential logic: FSM state and counters
    always @(posedge clk or posedge reset) begin
        if (reset) begin
            state <= IDLE;
            n_L <= 0; n_R <= 0;
            tap_L <= 0; tap_R <= 0;
            coeff_L_count <= 0; coeff_R_count <= 0;
            rj_L_value <= 0; rj_R_value <= 0;
            coeff_delay_L <= 0; coeff_delay_R <= 0;
            index_L <= 0; index_R <= 0;
        end else begin
            state <= next_state;

            case (state)
                LOAD_RJ: begin
                    rj_L_value <= rj_L_data;
                    rj_R_value <= rj_R_data;
                end

```

```
                LOAD_COEFF: begin
                    coeff_delay_L <= coeff_L_data[7:0];
                    coeff_delay_R <= coeff_R_data[7:0];
                end

                CALC_INDEX: begin
                    index_L <= n_L - coeff_delay_L;
                    index_R <= n_R - coeff_delay_R;
                end

                INCREMENT: begin
                    // LEFT loop logic
                    if (coeff_L_count < rj_L_value - 1)
                        coeff_L_count <= coeff_L_count + 1;
                    else begin
                        coeff_L_count <= 0;
                        if (tap_L < 15)
                            tap_L <= tap_L + 1;
                        else begin
                            tap_L <= 0;
                            if (n_L < 999)
                                n_L <= n_L + 1;
                        end
                    end

                    // RIGHT loop logic
                    if (coeff_R_count < rj_R_value - 1)
                        coeff_R_count <= coeff_R_count + 1;
                    else begin
                        coeff_R_count <= 0;
                        if (tap_R < 15)
                            tap_R <= tap_R + 1;
                        else begin
                            tap_R <= 0;
                            if (n_R < 999)
                                n_R <= n_R + 1;
                        end
                    end
                end
            endcase
        end
    end

    // Combinational FSM logic
    always @(*) begin
        // Default outputs
        clear = 0;
        clear_reg = 0;
        load_adder = 0;
        load_shift = 0;
        shift = 0;

        read_addr_rj_L = tap_L;
        read_addr_rj_R = tap_R;
        read_addr_coeff_L = coeff_L_count;
        read_addr_coeff_R = coeff_R_count;
        read_addr_data_L = 8'd0;
        read_addr_data_R = 8'd0;

        next_state = state;

        case (state)
            IDLE: begin
                if (en_fir) begin
                    clear = 1;
                    clear_reg = 1;
                    next_state = LOAD_RJ;
                end
            end
```

```
553
554                 LOAD_RJ: begin
555                     next_state = LOAD_COEFF;
556                 end
557
558                 LOAD_COEFF: begin
559                     next_state = CALC_INDEX;
560                 end
561
562                 CALC_INDEX: begin
563                     load_adder = 1;
564                     read_addr_data_L = index_L[7:0];
565                     read_addr_data_R = index_R[7:0];
566                     next_state = SHIFT_DATA;
567                 end
568
569                 SHIFT_DATA: begin
570                     load_shift = 1;
571                     next_state = WAIT_SHIFT_DONE;
572                 end
573
574                 WAIT_SHIFT_DONE: begin
575                     if (done_shifting)
576                         next_state = INCREMENT;
577                 end
578
579                 INCREMENT: begin
580                     if (n_L >= 999 && n_R >= 999)
581                         next_state = DONE;
582                     else
583                         next_state = LOAD_RJ;
584                 end
585
586                 DONE: begin
587                     next_state = IDLE;
588                 end
589             endcase
590         end
591
592     endmodule
593
594
595
596     module SignExtender (
597       input  wire [31:0] datain,
598       output wire [23:0] dataout_L,
599       output wire [23:0] dataout_R
600     );
601       // Sign-extend left half (bits 31:16) using bit 31
602       assign dataout_L = {{8{datain[31]}}, datain[31:16]};
603
604       // Sign-extend right half (bits 15:0) using bit 15
605       assign dataout_R = {{8{datain[15]}}, datain[15:0]};
606     endmodule
607
608
609     //adder is been updated to do add on both Left and Right
610     module Adder (
611       //Left side of the adder
612       input  wire [23:0] dataout_L,
613       input  wire [23:0] shift_outL,
614       input  wire        add_sub_L, // 0: add, 1: subtract
615       output wire [23:0] adder_outL,
616
617       //Right side of the adder
618       input  wire [23:0] dataout_R,
619       input  wire [23:0] shift_outR,
620       input  wire        add_sub_R, // 0: add, 1: subtract
621       output wire [23:0] adder_outR
```

```
);
    assign adder_outL = add_sub_L ? (shift_outL - dataout_L) : (shift_outL + dataout_L);
    assign adder_outR = add_sub_R ? (shift_outR - dataout_R) : (shift_outR + dataout_R);
endmodule



module ShiftRightRegister (
    input  wire        clk,
    input  wire        reset,
    input  wire        load,
    input  wire [23:0] adder_outL,
    input  wire [23:0] adder_outR,
    output reg  [39:0] shift_outL,
    output reg  [39:0] shift_outR,
    output reg         done_shifting
);

    reg [5:0] shift_count;
    reg       shifting;

    always @(posedge clk or posedge reset) begin
        if (reset) begin
            shift_outL     <= 40'd0;
            shift_outR     <= 40'd0;
            shift_count    <= 6'd0;
            shifting       <= 1'b0;
            done_shifting  <= 1'b0;
        end else begin
            done_shifting <= 1'b0;

            if (load) begin
                shift_outL[39:16] <= adder_outL;
                shift_outL[15:0]  <= 16'd0;
                shift_outR[39:16] <= adder_outR;
                shift_outR[15:0]  <= 16'd0;
                shift_count       <= 6'd0;
                shifting          <= 1'b1;
            end else if (shifting) begin
                shift_outL <= {1'b0, shift_outL[39:1]};
                shift_outR <= {1'b0, shift_outR[39:1]};
                shift_count <= shift_count + 1;

                if (shift_count == 6'd31) begin
                    shifting      <= 1'b0;
                    done_shifting <= 1'b1;
                end
            end
        end
    end

endmodule
```